Keagan Ryder

Milestone Two: Enhancement One

Software Design and Engineering

Brief Description of the Artifact

The artifact I selected for the Software Design and Engineering category is my Android Inventory Tracker application, originally developed in CS 360. This app is built in Android Studio using Java and SQLite. It allows a user to add items, store their quantities in a local database, and update or delete entries through a simple mobile interface. The original version of the project focused on basic mobile app structure, local data persistence, and a functional RecyclerView-based list of inventory items. For CS 499 Milestone Two, I revisited this artifact to enhance its overall design and engineering quality by adding login authentication, a logout flow, and an improved search feature for the inventory list.

Justification for Inclusion and Description of Improvements

I chose this artifact for my ePortfolio because it demonstrates real-world software engineering skills in a mobile context. It represents a complete, working application that integrates user interface design, local data storage with SQLite, and core app logic. Many professional roles involve building or maintaining mobile or client-facing applications, so being able to showcase a functioning Android app that I designed and enhanced is valuable evidence of my abilities.

Several specific components of this artifact showcase my skills and growth. First, I implemented a proper login authentication system using a users table in the SQLite database and simple credential validation logic. The app now supports creating accounts and logging in, and it securely remembers the logged-in user using SharedPreferences so that the login screen can be skipped on subsequent launches. I also added a logout button on the main inventory screen that clears the saved authentication state and returns the user to the login activity. This shows that I can design a basic authentication flow and manage user sessions, which is a common need in professional applications.

Second, I added a search bar and dedicated search button that filter the inventory list based on the item name. The RecyclerView adapter was refactored to maintain a full list of items and a visible, filtered list. When the user enters a query and presses Search, the adapter performs case-insensitive filtering. When the search bar is cleared and the Search button is pressed again, all items are shown. This demonstrates that I can integrate UI components with underlying data structures and implement a responsive, user-friendly search experience.

Third, I improved input validation and the overall data flow. The main activity now validates item names and quantities before inserting them into the database. If the quantity is invalid or negative, the app shows an appropriate message and prevents the insert. After successfully adding a new item, the app clears the input fields, clears the search text, and refreshes the list so that the new entry appears immediately. Together, these changes show my ability to refine a working codebase to make it more robust, user-friendly, and maintainable.

Course Outcomes and Outcome Coverage

In my Module One planning, I intended for this artifact to primarily address the outcome related to software engineering and design by demonstrating the ability to use sound techniques, tools, and practices to implement a computing solution. With the enhancements I have now completed, I believe I met that goal. I used structured database access, clear separation between the data layer (DatabaseHelper) and UI logic (activities and adapter), and standard Android patterns for storing user preferences and navigating between screens.

This enhancement also touches on other outcomes. The search feature and list handling relate to designing and evaluating computing solutions using appropriate data structures and algorithmic principles, since I had to think about how to store items, maintain both a full list and a filtered list, and handle user queries efficiently in memory. In addition, by improving the clarity of the interface and error messages, I have made progress toward the communication outcome, delivering a more coherent and understandable user experience. At this time, I do not need to change my original outcome coverage plan, but I can now more confidently say that the artifact supports both software design/engineering and data-structure-related outcomes.

Reflection on the Enhancement Process

The process of enhancing this app for Milestone Two helped me think more like a software engineer who is evolving an existing product rather than just finishing a one-time assignment. Implementing login authentication required me to consider how data is stored over the lifetime of the app, how to create and validate users, and how to maintain a logged-in state without forcing the user to re-enter credentials every time. I learned how to leverage

SQLiteOpenHelper and SharedPreferences together to create a simple but realistic authentication flow.

Adding the search feature and refining the RecyclerView adapter taught me more about separating concerns in UI components. Instead of directly binding the cursor to the adapter, I moved to a model where the adapter manages lists of Item objects and handles filtering internally. This made the code easier to reason about and allowed me to add features like “empty search shows all results” more cleanly. I also learned how small decisions in data modeling and adapter design can significantly affect both usability and maintainability.

One of the main challenges I faced was managing changes to the database schema and making sure the app worked correctly across rebuilds. When I modified the tables to support user accounts and seeded a default admin user, I had to adjust the database version and ensure the upgrade path was handled so that the app would recreate the tables correctly on devices where an older version of the database already existed. Debugging issues like failed inserts helped me better understand how SQLite behaves and how to design defensive code that checks return values and handles failures gracefully.

Overall, enhancing this artifact strengthened my understanding of Android app architecture, local data storage, and user-centered design. It also showed me how to incrementally add features such as authentication and search while keeping the codebase organized. These are the same types of skills I will need when working on production systems in a professional setting, which is why this artifact is a strong fit for my ePortfolio in the software design and engineering category.